# Métodos Numéricos

*Máster en Computación Gráfica, Realidad Virtual y Simulación*

U-tad — Curso 24/25

Proyecto Final

## § Instrucciones §

- El código debe organizarse en scripts de Python (`.py`) debidamente documentados para cada etapa.
- No está permitido el uso de código procedente de referencias o modelos de lenguaje (LLMs) como ChatGPT o Github Copilot.
- **Se debe incluir un archivo `README.md`** que explique la estructura del proyecto, cómo ejecutar los scripts e instrucciones para la interacción del usuario con la simulación en cada etapa (si la hubiera).
- El archivo `README.md` debe contener una sección de Referencias donde se detallen todos los recursos utilizados para la resolución del proyecto. En caso de consulta (no para generar código) a modelos de lenguaje (LLMs) , indicar los prompts utilizados y las respuestas obtenidas.
- Todos los ficheros deben entregarse comprimidos en un fichero `.zip`.
- La calificación máxima del proyecto final es un 10. El proyecto final deberá defenderse ante el profesor.

**Uso de Librerías**

Se permite el uso de las librerías estándar de Python y la librería `Taichi Lang`. Se puede usar `NumPy` para tareas auxiliares, pero se espera que la lógica central de la simulación se implemente en Taichi. El uso de otras librerías requiere autorización previa.

## Simulación de Fluidos en Tiempo Real para Videojuegos

*Implementación Guiada*

Este proyecto tiene como objetivo la implementación de un simulador de fluidos en tiempo real basado en el paper "*Real-Time Fluid Dynamics for Games*" de Jos Stam, utilizando Taichi Lang. Se busca lograr una simulación de "Stable Fluids" mediante la implementación de los conceptos clave del paper.

## Etapas 0 – 2: Moviendo Densidades

En las primeras etapas, estableceremos las bases visuales de la simulación. Comenzaremos explorando el movimiento de la densidad, que representará visualmente el fluido como "humo" o "colorante", en un campo de velocidad estático. En cada paso de tiempo, la densidad evolucionará a través de tres operaciones fundamentales: primero, se añadirán nuevas fuentes de densidad al sistema. Segundo, la densidad se difundirá, simulando la dispersión natural. Tercero, la densidad existente se moverá a través del campo de velocidad predefinido. Este enfoque inicial nos permite centrarnos en la manipulación de la densidad antes de incorporar la dinámica de la velocidad.

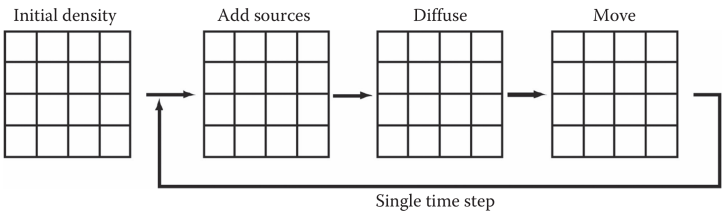


**Figura 1:** Esquema de actualización de malla de densidad en un paso de tiempo fijo

## Etapa 0: Densidad – Añadir Fuentes

En esta etapa inicial, el objetivo es implementar la capacidad de **añadir fuentes de densidad** a la simulación. Esto permitirá visualizar el fluido y observar cómo se comporta en las etapas posteriores. El código del fichero `etapa0.py` implementa la posibilidad de añadir fuentes de forma interactiva utilizando el botón derecho del ratón.

*Nota sobre la interfaz gráfica (GUI): Si experimentas problemas al utilizar la nueva interfaz de usuario GGUI de Taichi Lang (basada en Vulkan) puedes utilizar la versión anterior de la GUI. Para facilitar el uso de la versión anterior, se proporciona el código de inicio alternativo `etapa0_no_vk.py`.*

## Etapa 1: Densidad – Difusión [4 puntos]

En esta etapa, el objetivo es implementar la **difusión de la densidad** y explicar el método numérico implementado. Este proceso simula cómo la densidad se dispersa, suavizando las concentraciones en el fluido.

1. **Implementar la función `diffuse` (Método de Jacobi).** Crea una función `diffuse` que aplique el paso de difusión. Dentro de ella, utiliza el **método iterativo de Jacobi** para resolver la ecuación de difusión de forma discreta, necesitando un kernel de Taichi (`jacobi_iter`) para realizar cada iteración de forma paralela. **Experimenta con un número fijo de iteraciones, por ejemplo, entre 40 y 150 iteraciones para cada paso de tiempo.**
2. **Implementar Condiciones de Frontera con Grid Extendido.** Utiliza un **grid extendido** para facilitar las **condiciones de frontera**. Implementa un kernel de Taichi (`set_boundaries`) para actualizar las celdas frontera, copiando el valor de la celda interna correspondiente. En las esquinas, utiliza el promedio de las celdas vecinas. Esta función se debe llamar tras añadir fuentes de densidad y tras cada iteración de Jacobi.

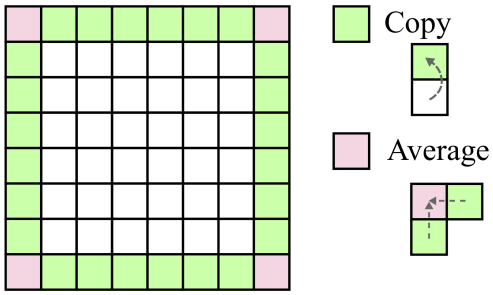


**Figura 2:** Condiciones frontera densidad: continuidad

**Vídeo-Explicación**

Vídeo explicativo (**máximo 4 minutos**) detallando el **método iterativo de Jacobi para la resolución de la ecuación de difusión**.
En el vídeo se debe:

- Explicar de forma concisa y clara el método iterativo de Jacobi implementado, destacando su funcionamiento general y su relevancia para la simulación de la difusión en fluidos.
- Mostrar claramente el paralelismo entre la notación matemática de las iteraciones de Jacobi y la estructura del código implementado en el kernel `jacobi_iter` de Taichi, demostrando así la comprensión de la implementación.
- Mostrar la simulación de fluidos funcionando con el método de difusión de Jacobi implementado y explicar cómo se observa visualmente el efecto de la difusión en el campo de densidad.

*La comprensión demostrada y la claridad en la Vídeo-Explicación corresponderá al 50% de la calificación de la etapa. La ausencia de Vídeo-Explicación implicará un 0 en la calificación de la etapa.*

## Etapa 2: Densidad – Advección [3 puntos]

En esta etapa, el objetivo es implementar la **advección de la densidad**. Este proceso simula cómo la densidad es transportada por el campo de velocidad del fluido.

1. **Implementar la función `advect` (Advección Semi-Lagrangiana con Interpolación Bilineal).** Crea un kernel de Taichi `advect` que aplique el paso de advección usando el método semi-Lagrangiano. Este kernel deberá usar una función auxiliar `bilerp` para realizar la interpolación bilineal necesaria.
2. **Inicializar Campo de Velocidades.** En la función `init`, inicializa el campo de velocidad con un patrón simple. *Por ejemplo, el campo de velocidades podría ser inicializado de manera que haya una franja horizontal en el medio del dominio con velocidades de izquierda a derecha y dos franjas horizontales (una arriba y otra abajo) con velocidades de derecha a izquierda.*

**Vídeo Explicación**

Vídeo explicativo (**máximo 3 minutos**) detallando el **método de Advección Semi-Lagrangiana con Interpolación Bilineal**.
En el vídeo se debe:

- Explicar de forma concisa y clara el método de Advección Semi-Lagrangiana implementado, destacando su relevancia para simular el transporte de la densidad en fluidos.
- Explicar claramente el uso de la Interpolación Bilineal dentro del método de Advección Semi-Lagrangiana, demostrando así la comprensión de su rol.
- Mostrar la simulación de fluidos funcionando con el método de Advección Semi-Lagrangiana implementado y explicar cómo se observa visualmente el efecto de la advección en el campo de densidad.

*La comprensión demostrada y la claridad en la Vídeo-Explicación corresponderá al 50% de la calificación de la etapa. La ausencia de Vídeo-Explicación implicará un 0 en la calificación de la etapa.*

## Etapas 3 – 5: Evolución de la Velocidad

Ahora extenderemos la simulación para incluir la evolución del campo vectorial de la velocidad. La velocidad ya no será estática sino evolucionará influenciada por fuerzas externas que simularán la interacción del usuario o el entorno, y por la viscosidad del fluido, que provocará la difusión de la velocidad. Además, la propia velocidad del fluido se auto-advectará, es decir, el fluido en movimiento transporta su propia velocidad. Para simular la velocidad de forma precisa, también debemos asegurar que el flujo se mantenga incompresible, un aspecto que se abordará en la última etapa. En resumen, la evolución de la velocidad en cada paso de tiempo implica: la adición de fuerzas, la difusión de la velocidad, la auto-advección de la velocidad más un paso de proyección para garantizar la incompresibilidad del fluido.

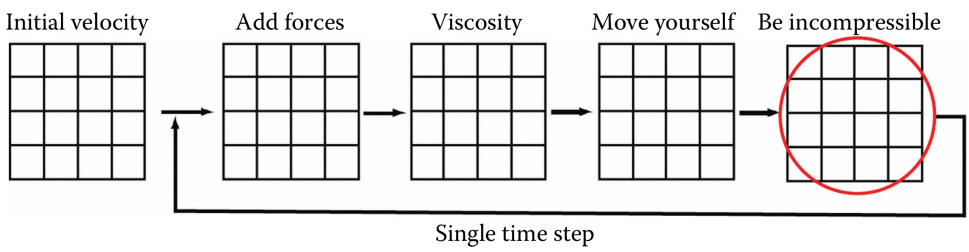


**Figura 3:** Esquema de actualización de malla de velocidad en un paso de tiempo fijo

## Etapa 3: Velocidad – Añadir Fuerzas [1.5 puntos]

En esta etapa, se introduce la capacidad de **añadir fuerzas externas** al fluido, permitiendo la interacción del usuario para generar movimiento. La velocidad inicial estática se elimina.

1. **Eliminar Velocidad Inicial:** Modifica `init` para inicializar el campo de velocidad a cero, eliminando la inicialización estática de la etapa anterior.
2. **Capturar Input para Fuerzas:** Implementa una **nueva interacción** en `main` para capturar el input del usuario que definirá la fuerza a aplicar (posición, dirección). *Se recomienda utilizar el movimiento del ratón con el botón izquierdo presionado para generar un vector de fuerza basado en el desplazamiento, pero se pueden implementar otras interacciones.* Añade los datos necesarios (posición, dirección, activación) al array `input_data`.
3. **Implementar la función `add_forces` (Aplicar Fuerzas al Campo de Velocidad):** Implementa un mecanismo para aplicar fuerzas al campo de velocidad. Esto se puede hacer mediante un nuevo kernel `add_forces` similar a `add_sources` o modificando `add_sources` para que maneje tanto campos escalares (densidad) como vectoriales (velocidad) en base a `input_data`.

**Vídeo-Explicación**

Vídeo explicativo (**máximo 1.5 minutos**) mostrando la **interacción implementada y el resultado**.
En el vídeo se debe:

- Explicar brevemente el código de la interacción implementada para añadir fuerzas.
- Mostrar la simulación de fluidos interactuando para aplicar fuerzas externas y explicar cómo se observa visualmente el efecto de la advección en el campo de densidad.

*La comprensión demostrada y la claridad en la Vídeo-Explicación corresponderá al 50% de la calificación de la etapa. La ausencia de Vídeo-Explicación implicará un 0 en la calificación de la etapa.*

## Etapa 4: Velocidad – Difusión y Advección [1.5 puntos]

En esta etapa, se aplicarán los mecanismos de **difusión** y **advección** al propio campo de velocidad. Esto introducirá el efecto de la **viscosidad** y la **auto-advección** (el fluido transporta su propia velocidad). Además, se implementarán las **condiciones de frontera** para el campo vectorial de las velocidades, que serán claves una vez se aplique la incompresibilidad.

1. **Adaptar `diffuse` y `advect` para Campos Vectoriales:** Adaptar, si fuera necesario, `diffuse`/`jacobi_iter` y `advect`/`bilerp` para que funcionen correctamente cuando se aplican a campos vectoriales.
2. **Implementar Condiciones Fronteras Reflectantes:** Crear un nuevo kernel `set_boundaries_reflective` que implemente las condiciones de frontera reflectantes: invirtiendo la velocidad en la dirección normal a la frontera (componente `x` en paredes verticales, componente `y` en horizontales) y copiando la tangencial (componente `y` en paredes verticales, componente `x` en horizontales). Esta función se debe llamar tras añadir fuerzas, tras advectar la velocidad y tras cada iteración de Jacobi dentro de la difusión de la velocidad.

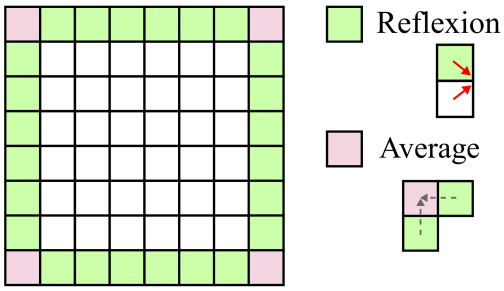


**Figura 4:** Condiciones frontera velocidad: reflejadas

**Vídeo-Explicación**

Vídeo explicativo (**máximo 1.5 minutos**) detallando la implementación de la difusión, advección y condiciones de frontera al campo de velocidades.
En el vídeo se debe:

- Explicar brevemente los cambios realizados y la implementación de las condiciones frontera reflectantes.
- Mostrar la simulación de fluidos con viscosidad y auto-advección aplicadas a la velocidad.

*La comprensión demostrada y la claridad en la Vídeo-Explicación corresponderá al 50% de la calificación de la etapa. La ausencia de Vídeo-Explicación implicará un 0 en la calificación de la etapa.*

## Etapa 5: Proyección para Incompresibilidad [Opcional, 2 puntos]

El último paso clave del método "Stable Fluids" es asegurar que el campo de velocidad sea **libre de divergencia**, garantizando así la incompresibilidad del fluido. Este paso solo se aplica al campo de velocidad y consiste en **proyectarlo** sobre el espacio de campos vectoriales libres de divergencia. El proceso, detallado en el paper de Jos Stam, se puede resumir en tres sub-pasos conceptuales aplicados al campo de velocidad $w$ (resultado de la advección):

1. **Calcular Divergencia:** Computar el campo escalar de divergencia $d = \nabla \cdot w$, que mide la expansión o compresión local del campo $w$.
2. **Resolver Ecuación de Poisson:** Encontrar un campo escalar $p$ (presión) que satisfaga la ecuación de Poisson $\nabla^2 p = d$. Esto se resuelve numéricamente, típicamente usando un método iterativo estable como Gauss-Seidel o Jacobi.
3. **Restar Gradiente:** Calcular el campo de velocidad final $u$, libre de divergencia, restando el gradiente de presión $\nabla p$ del campo original: $u = w - \nabla p$.

**Implementación:**

1. **Nuevos Campos y Kernels:** Introduce los campos escalares necesarios (`divergence`, `pressure`) y los kernels para implementar los tres sub-pasos (cálculo de divergencia, iteración de Jacobi/Gauss-Seidel para Poisson, y resta del gradiente) usando diferencias finitas (*ver detalles en el paper*).
2. **Función `project`:** Agrupa la lógica de los tres sub-pasos en una función Python `project()`. Asegúrate de aplicar las condiciones de frontera (`set_boundaries` para `divergence` y `pressure`, `set_boundaries_reflective` para la velocidad final) en los momentos adecuados.

**Vídeo-Explicación**

Vídeo explicativo (**máximo 4 minutos**) detallando el **paso de proyección y su efecto**.
En el vídeo se debe:

- Explicar, al nivel de comprensión alcanzado, los tres sub-pasos implementados (cálculo de divergencia, solución de Poisson, resta de gradiente).
- Mostrar la simulación funcionando con la proyección activada. Destacar visualmente cómo la proyección genera flujos más realistas y estables, con vórtices característicos y cómo el fluido ahora respeta las fronteras sin atravesarlas.

*La comprensión demostrada y la claridad en la Vídeo-Explicación corresponderá al 50% de la calificación de la etapa. La ausencia de Vídeo-Explicación implicará un 0 en la calificación de la etapa.*

## Extras [Opcional, hasta 4 puntos]

Adicionalmente, se pueden implementar una o varias de las siguientes extensiones para mejorar la simulación o añadir nuevas funcionalidades. La puntuación dependerá de la complejidad, la calidad de la implementación y la video-explicación.

1. **Visualización del Campo de Velocidad:** Implementar una visualización opcional del campo de velocidad mediante glifos (líneas o flechas), activable/desactivable con una tecla.
2. **Fronteras Interiores (Obstáculos):** Introducir obstáculos simples (círculo o rectángulo estático) en el dominio. Modificar los pasos de advección, difusión y proyección para que respeten estas fronteras internas.
3. **Densidad de Color (RGB):** Modificar los campos de densidad y los kernels relacionados para manejar vectores de 3 componentes (RGB) en lugar de un escalar y añadir densidades de diferentes colores.
4. **Extensión a 3D:** Adaptar toda la simulación para funcionar en un grid tridimensional (requiere cambios significativos en casi todas las etapas).
5. **Tu Propia Idea:** Implementar cualquier otra funcionalidad o mejora que consideres interesante.

**Vídeo-Explicación (Extras)**

Vídeo explicativo (**máximo 4 minutos**) mostrando y explicando **las funcionalidades extra implementadas**. En el vídeo se debe:

- Mostrar claramente cada una de las funcionalidades extra implementadas.
- Explicar brevemente los cambios realizados en el código para lograr cada extra.
- Comentar el efecto visual o funcional obtenido con cada mejora.